San Jose State University

EE174: Analog Peripheral for Embedded Systems

Harkiret Singh (014609374)

Sébastien Trocherie (016040193)

Professor Tan Nguyen

Abstract

For our final project we decided to build a drone. We chose a drone because it includes multiple fields of electronics (power electronics, communications, control systems). To make this project work we needed to design a frame for the drone, choose the right components, solder our mainboard and write the code.

Procedure and Method

We started out by 3D designing our frame, this process was done in Fusion 360 3D design software. The aim of designing our quadcopter was to make the frame as small as possible since this means less weight and in terms we need less lifting force.

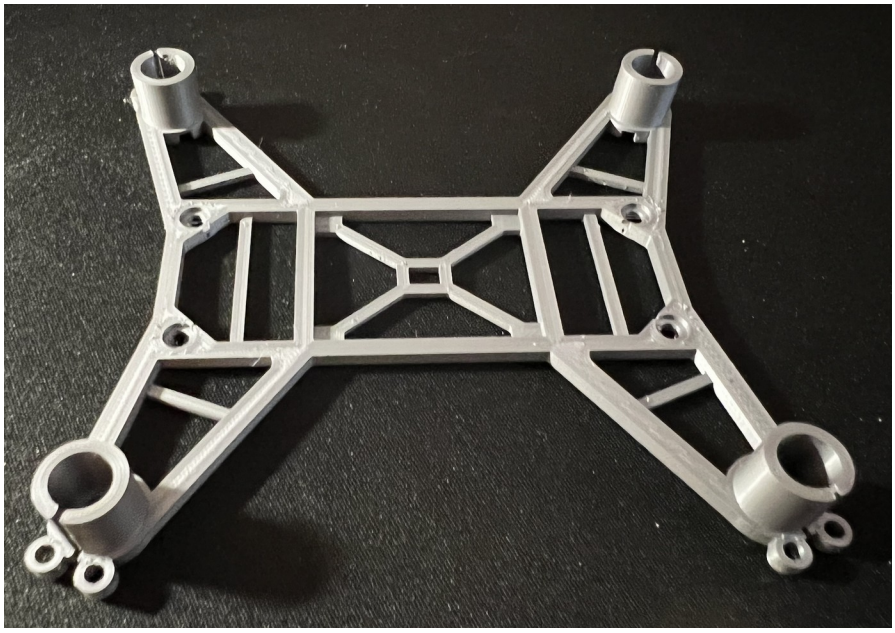


Figure 1: 3D printed frame

We designed the frame in such a way that it would fit a wide variety of DC drone engines. We decided to get brushed DC motors because they’re cheaper and still generate enough lifting power to get our drone in the air.

The next step was choosing the DC motors, the frame fits 8520 motors which means the dimensions of the engines must be 8.5mm x 20mm. This is one consideration that's important for the construction of the drone but there is a lot more to choosing the right motors, the following points were all taken into consideration:

- Type - there are brushed DC and brushless AC motors. Nano quadcopters are usually based on brushed DC motors as they're smaller and easier to control without the need of extra AC controllers. However they have a lot less thrust and cannot be used for larger quadcopters.
- Can diameter - the 3D design was made for motors with 8.5mm diameter. The design would have to be altered for a different diameter.
- Max Static Thrust or simply Thrust - defines how much weight can the motors keep in the air or basically - how heavy your quadcopter can be. Propeller type have to be defined and often thrust vs current curve (often called performance curve) is supplied
- Weight - the weight of a motor, which will be added to the total weight of the quadcopter.
- Load current - defines what current is being drawn by a motor when a specified voltage is applied using specified propellers. Note that without the propellers attached this current would drop to really small values
- Lifetime rating - this defines how long should the motors run without failing, for cheap engines we expect a lifetime of 5-6h
- Speed - fast speed will give you faster flights, however with increased currents, you & I need better batteries.

Taking all these points into consideration the 8520 DC brushed motor was the best fit for this project since they ticked pretty much every box but mostly they were in stock and relatively cheap. The only big downside is that there is no datasheet available.

Arguably the most important part of any electronics project is the power supply, we want a power supply that is as stable as possible since it will be feeding both the microcontroller, sensors and motors.

Since the drone will move around there is no other way of supplying power to the circuit then to use a battery. Choosing adequate batteries is very important as wrong batteries will not allow the motors to draw enough current to light up the quadcopter and on top of that there will be huge voltage drops, which will keep interfering with the electronics.

To choose the right battery, the following points were taken into consideration:

- Type - there are many battery types such as li-po, li-on and even these come in shades. Rather recently Graphene based batteries became available, which can withstand higher currents and have higher capacity density. Of course the downside is the price!
- Capacity (measured in mAh or Wh) - will determine how much energy is stored inside the battery. The larger the capacity, the longer the quadcopter will run on a single charge. This will be proportional to the size and weight of the battery and will also determine how much current can be drawn from and into the battery.
- Max allowed discharge (burst) and average discharge (constant) rate (C) - the first determines peak currents e.g. when quadcopter starts accelerating, while the latter determines normal operation current e.g. quadcopter is kept constant in the air. There is often a rule of thumb that multiplication of capacity and the discharge rate will give the current that the batteries can supply. To be on the safe side we also need to add 20% of

safety margin - we do not want the batteries to explode do we? Thus for example, if you had a battery with 200 mAh and 20 C of average/constant discharge, then 200mAh * 25C * 80% = 4A. Thus on average such a battery can supply 4A without issues. However, this is just a rule of thumb, when very high currents are involved, we want the discharge rate to be way higher, independently to the battery capacity

- Weight (g) - larger capacity batteries weigh more, thus need to find some which provide enough flying time but still provide good flying performance.

Let's calculate which batteries to choose and how long the batteries will last. In our case the motors draw hundreds of times less current than the electronics, thus when calculating the required discharge rate we only need to care about the motor currents. Total max load current = 2.75A * 4 = 11A. Good battery capacity should range from 150-350mAh and I chose a battery with a little extra: 650mAh. To simplify calculations I will use 0.65Ah. This means I need 11A / 0.65Ah = 16.9C of max discharge rate. From this follows that on a single charge they should last at least for 0.65Ah * 60min / 11A = 3.54 min. It seems like the numbers are reasonable.

Now we still need a microcontroller that will process all our signals and control the engines. This drone is a proof of concept and will be used to further expand our knowledge in electronics. That’s why we chose to use the Arduino platform. The Arduino platform is great for prototyping since there’s a lot of projects and open source knowledge available.

Choosing an Arduino that best fits our project wasn’t easy, there are a lot of different models and boards available but in the end we decided to choose the Arduino Nano 33 BLE. We've chosen

this controller because of its built- in BLE module and PWM output functionality. We can map the internal PWM generator to anyone of the 14 digital output pins.

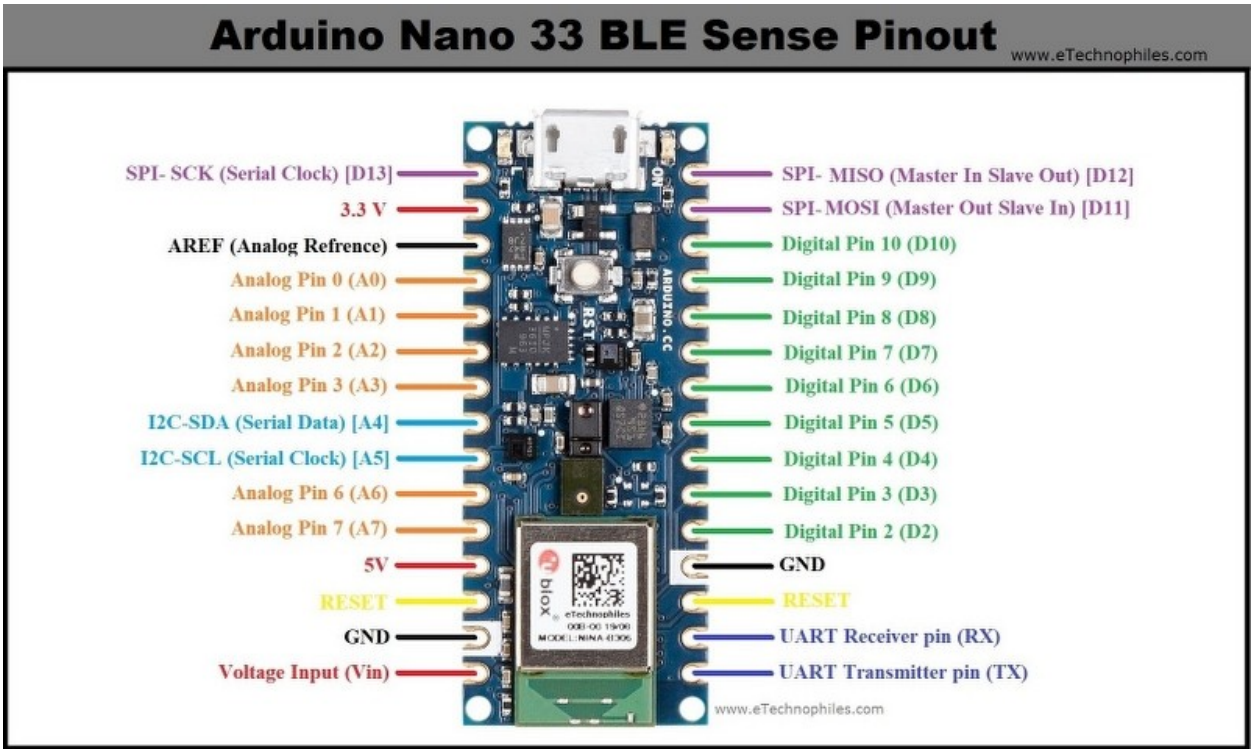


Figure 2: Arduino Nano 33 BLE pinout

We need the internal PWM generator because we will use a PWM signal to control the speed of our DC motors, by changing the duty cycle of our PWM signal we can reduce the average voltage of the motors which in terms means they will turn more slowly. Of course we can't connect the motors directly to the GPIO of our Arduino Nano since this controller can only supply ~20mA per GPIO pin.

A work around for this problem is using mosfets, the positive side of the motors will be directly connected to the battery and the negative side is connected to the mosfet instead of directly to ground. The mosfet acts like a switch and will cut the connection and thus the flow of current to ground.

For this project we're going to need 4 mosfet transistors also known as switches. They will be used together with PWM to supply the current to the motors in pulses. Choosing them might be tricky and there are a couple of important points when when choosing one for out application:

- Maximum drain current it can supply (Id max), which in this case should be around 3A to support the motor which can draw around 2.75A.
- The Vgs threshold voltage, which has to be low, maybe somewhere around 1V. If too large a threshold is chosen Arduino won't be able to turn the MOSFETs on enough to supply the required amount of current. It is always best to also check the drain current (Id) dependency on the threshold voltage (Vgs threshold) as each transistor will have a different response.
- Drain to sink resistance when MOSFET is fully turned on (Rds (on)). This is provided under a certain gate to sink bias Vgs. Normally expect this to be around 0.032Ohm, however the smaller the better. If you choose some MOSFET with Rds (on) equal to 0.3Ohm or so, due to motors running at 3A the voltage drop over the MOSFET will be 0.9V, thus motors will not get sufficient voltage drop over them.

Moreover, we implemented a reverse battery protection system using a P-Mosfet. This was done instead of a diode since diode's tend to waste a lot of power. Since we wanted to save power dissipation we used a P-Mosfet. If a battery is fitted with opposite terminals, large amounts of currents will be delivered from the lipo battery, causing damage to the Arduino nano and the P-Mosfets.

Usage of Inertial Measurement Unit(IMU) - MPU 6050

Essentially IMU is used to calculate the angle between the quadcopters each axis and the ground. Normally IMU used inside quadcopters must have at least two elements - an accelerometer and a

gyroscope. For the project I used a very popular and cheap MPU6050 IMU, which includes a 3-axis Gyroscope and a 3-axis Accelerometer.

Since we had an issue implementing the MPU6050 on the Nano drone, we replicated its functionality on a beadboard with LED's instead of brushless motors. Below is a picture of the breadboard representation.

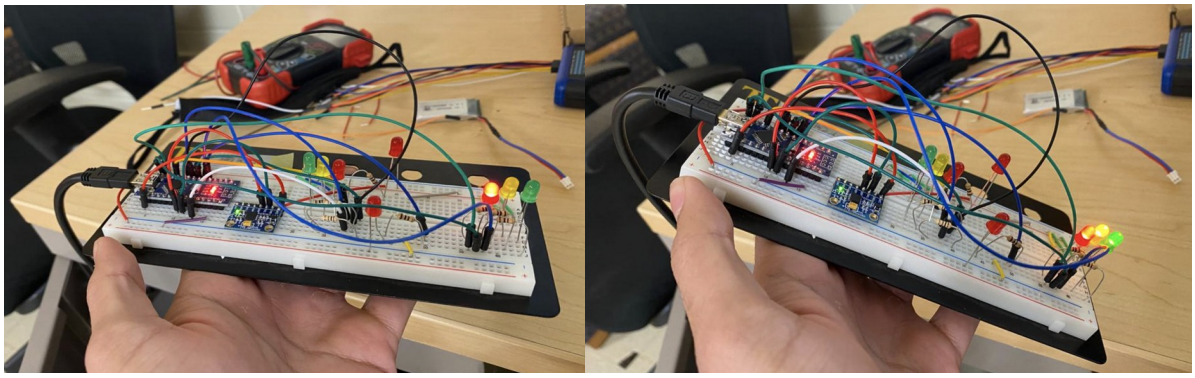


Figure 3: MPU 6050 concept replicating gyroscope feature of the Nano drone

The reason we couldn't integrate the MPU6050 sensor on the Nano drone was the 5V output of the Arduino arduino couldn't provide steady 5V voltage as it constantly dropped. This was due to an issue within the Arduino itself.

Program

Since we couldn't get the drone to fully work due to the reasons discussed above we decided to write the code in 3 separate parts. The first part is the BLE communication where we show that it is possible to communicate with the Arduino microcontroller over Bluetooth Low Energy. The second program will implement the I²C communication between the Arduino and the MPU6050. That program will show the working of the gyroscope as well as the PID algorithms. The final program will implement the PWM functionality and drive the motors at different speeds.

1) BLE program

BLE stands for Bluetooth Low Energy and is a new Bluetooth technology introduced in the new Bluetooth 4.0 standard. To keep it simple: BLE has a GATT (Generic Attribute Profile) which defines the way 2 BLE devices communicate with each other.

The GATT gets defined further by profiles, services and characteristics, we will make our own service and characteristic that will contain all our data we want to transfer. Each service and characteristic has a unique identifier called UUID, there are 16 and 128 bit UUID's. The 128 bit UUID are reserved for prototyping and can be freely used, we will create one for this project.

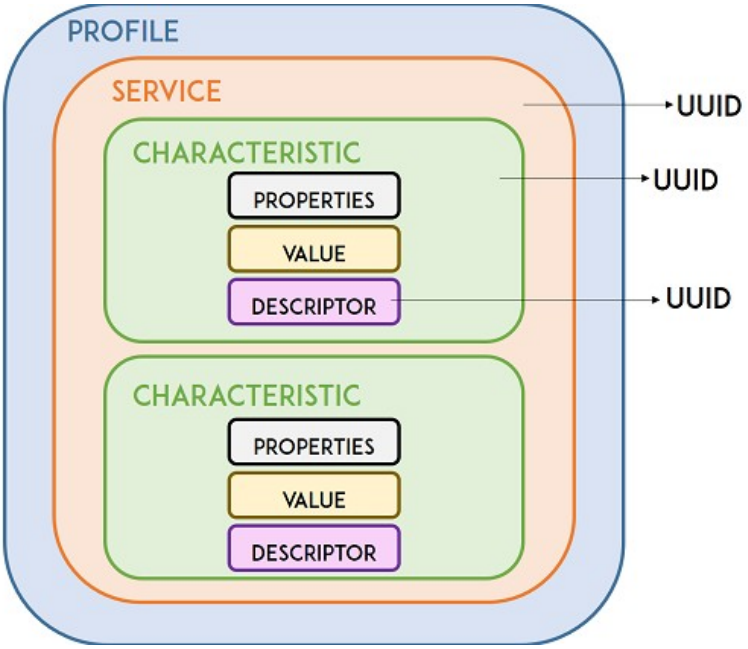


Figure 4: GATT database build-up

Now that we understand the working of BLE we can build our program, I will only go over the BLE parts of the program since any other part of the program is considered basic knowledge.

```
#include <ArduinoBLE.h>

const char* deviceServiceUuid = "faa94135-6303-4274-a2ce-8fa28849ac9c";
const char* deviceServiceCharacteristicUuid = "a6671089-7187-4a45-adee-6020212f014d";

int gesture = -1;

const int redPin = 22;
const int greenPin = 23;
const int bluePin = 24;

BLEService gestureService(deviceServiceUuid);
BLEByteCharacteristic gestureCharacteristic(deviceServiceCharacteristicUuid, BLERead | BLEWrite);
```

The first part of the program will include the BLE library we need to control the BLE module on the Arduino. Next we define the 128 bit service and characteristic which will be used for our communication. The gesture variable will hold the received value sent from our BLE app. Next we add our BLE service and characteristic to the GATT database of the Arduino, the characteristic is both read and write.

```
BLE.setLocalName("Drone");
BLE.setAdvertisedService(gestureService);
gestureService.addCharacteristic(gestureCharacteristic);
BLE.addService(gestureService);
gestureCharacteristic.writeValue(-1);
BLE.advertise();

Serial.println("Nano 33 BLE (Peripheral Device)");
Serial.println(" ");
```

The next part of the program does some more initialization of our BLE stack, we set a local name to ‘Drone’ so we can easily recognize it when we scan for BLE devices. We set both our characteristic and service and we write the value -1 to our characteristic. This is so we can easily know when a value is received from our phone.

Once all the initialization is done we can start advertising. During BLE advertisement, a BLE Peripheral device transmits the same packet on the 3 advertising channels, one after the other. A Central device scanning for devices or beacons will listen to those channels for the advertising packets, which helps it discover devices nearby.

```
BLEDevice central = BLE.central();
Serial.println("- Discovering central device...");
delay(500);

if (central) {
  Serial.println("* Connected to central device!");
  Serial.print("* Device MAC address: ");
  Serial.println(central.address());
  Serial.println(" ");

  while (central.connected()) {
    if (gestureCharacteristic.written()) {
      gesture = gestureCharacteristic.value();
      writeGesture(gesture);
    }
  }
```

This part of the code initializes the Arduino as a central device in the BLE communication meaning we've data to send to a BLE server or we want to receive data from the BLE server. The first if function will only be true if we are successfully connected to the BLE server, we then proceed to print the BLE address of the connected device.

While we stay connected the Arduino will continuously check if our custom characteristic gets changed in value, when this happens, we read the characteristics value and go to the "writeGesture" function.

The function can be seen on the next page and just receives the red green and blue value from our app, when the blue value gets received we send the value to the RGB leds.

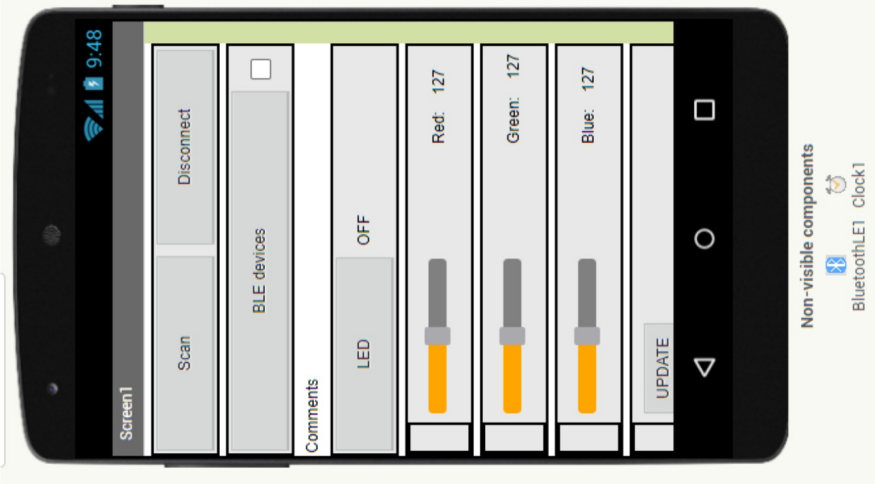
```
void writeGesture(int gesture) {
  static int count = 0;
  static int red = 0;
  static int green = 0;
  static int blue = 0;

  count++;

  if(count == 1)
  {
    red = gesture;
  }
  if(count == 2)
  {
    green = gesture;
  }
  if(count == 3)
  {
      blue = gesture;

      analogWrite(redPin, red);
      analogWrite(greenPin, green);
      analogWrite(bluePin, blue);
```

The app to connect to the Arduino and write values to it is made in the MIT app inventor studio environment. To build this app, we based ourselves on the following video explaining how to drive motors over BLE: [Arduino Bluetooth Motor Controller // Bluetooth Low Energy BLE and]



[MIT App Inventor for Android]

Figure 5: Screenshot of application GUI

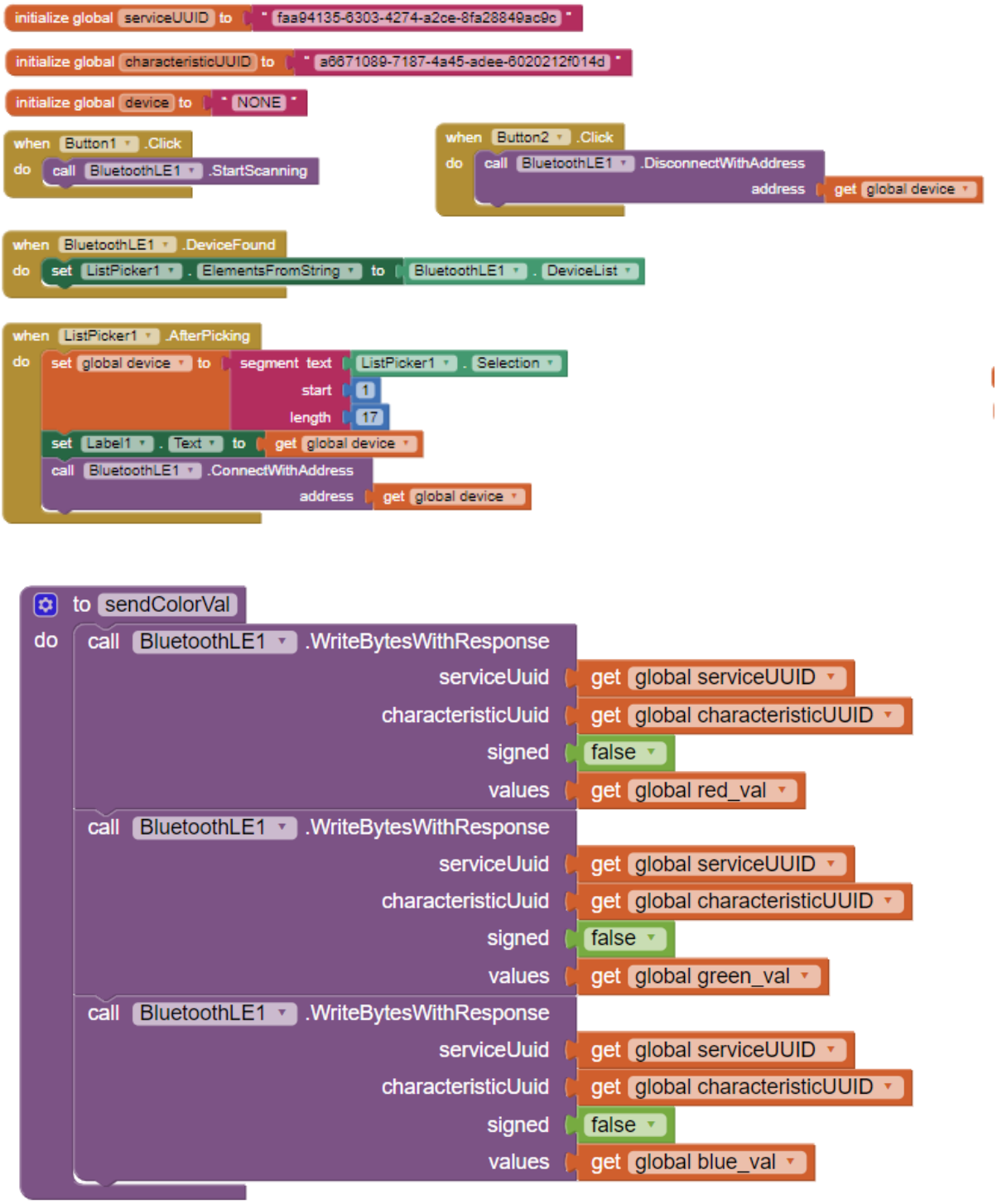


Figure 6:Programming code blocks

These are the 2 most important blocks of code in the app responsible for connecting to the BLE application and sending the values of our characteristic over BLE.

The PID controller of the drone takes values from the MPU 6050's gyroscope and accelerometer. It uses its values to come up with a control system.

```
// Define Variables we'll be connecting to
double pitchSetpoint, pitchInput, pitchOutput;
double rollSetpoint, rollInput, rollOutput;

//Define the aggressive and conservative Tuning Parameters
double consKp = 1, consKi = 0.05, consKd = 0.25;
PID pitchPID(&rollInput, &rollOutput, &rollSetpoint, consKp, consKi, consKd, DIRECT);
PID rollPID(&pitchInput, &pitchOutput, &pitchSetpoint, consKp, consKi, consKd, DIRECT);
```

As seen within the code above, the code is represented by the roll and pitch. These are the angles of the drone in regards to the x and y axis respectively. The pitchPID and rollPID takes in values for the functions and uses that as the variables.

```
  pitchPID.SetOutputLimits(-20, 20);
  rollPID.SetOutputLimits(-20, 20);

  targetSpeed[0] = -10;
  targetSpeed[1] = -10;
  targetSpeed[2] = -10;
  targetSpeed[3] = -10;
}

void changeTargetSpeed(int delta) {
  targetSpeed[0] += delta;
  targetSpeed[1] += delta;
  targetSpeed[2] += delta;
  targetSpeed[3] += delta;
}

void loop() {

  char bluetoothVal = 'a';

  if (Serial.available()) {
    bluetoothVal = Serial.read();
  }

  if (bluetoothVal == 'u') {
    changeTargetSpeed(1);
  }

  if (bluetoothVal == 'd') {
    changeTargetSpeed(-1);
  }
```

The target speed is set to -10, as the value ‘u’ is received, which means ‘up’, it reduces the target speed while as the BLE module receives ‘d’ which is ‘down’, the number gets larger. This is then controlled by the PIF controller.

Problems encountered:

One of the major issues we encountered within the building the drone project was an issue we had within the battery. The load current from the motors was too much which caused a voltage drop over the internal resistance of the battery. This caused the battery voltage to drop to 1.8V the Arduino Nano 33 BLE’s brown out voltage is 2.6V. This means that if the battery supply voltage dipped below 2.6V, the Arduino Nano 33 BLE would reset itself. The battery voltage spike can be seen as below in figure 7.

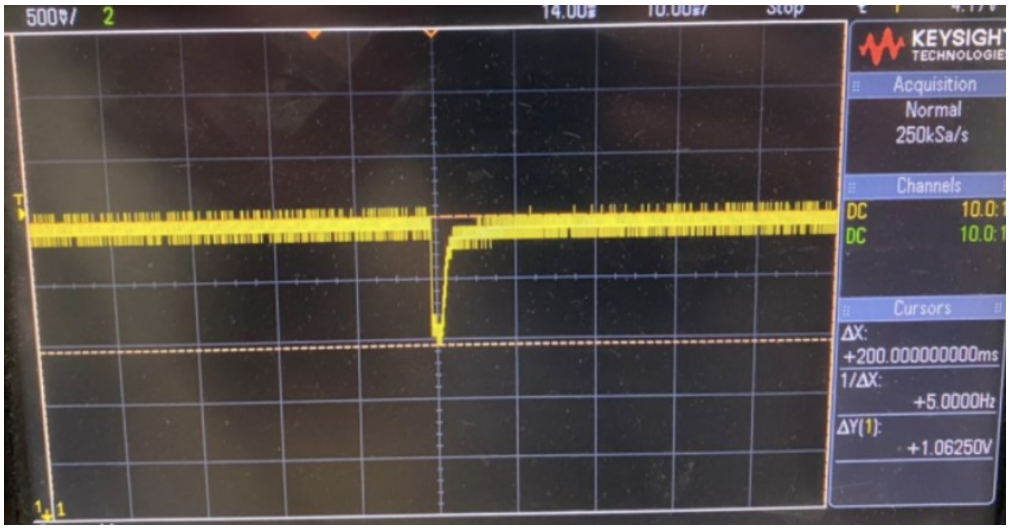


Figure 7: Voltage of battery spike due to too much load

Bill of materials:

Arduino Nano 33 BLE : $27.29

Connectors & ToF sensor: $35

Battery: $20.76

Motors : $27.32

Propellers: $16.38

Components from Mouser: $20.98

Components from Protosupplies: $23.40

Components from Anchor: $13.53

The total cost of this project was ***$184.66***

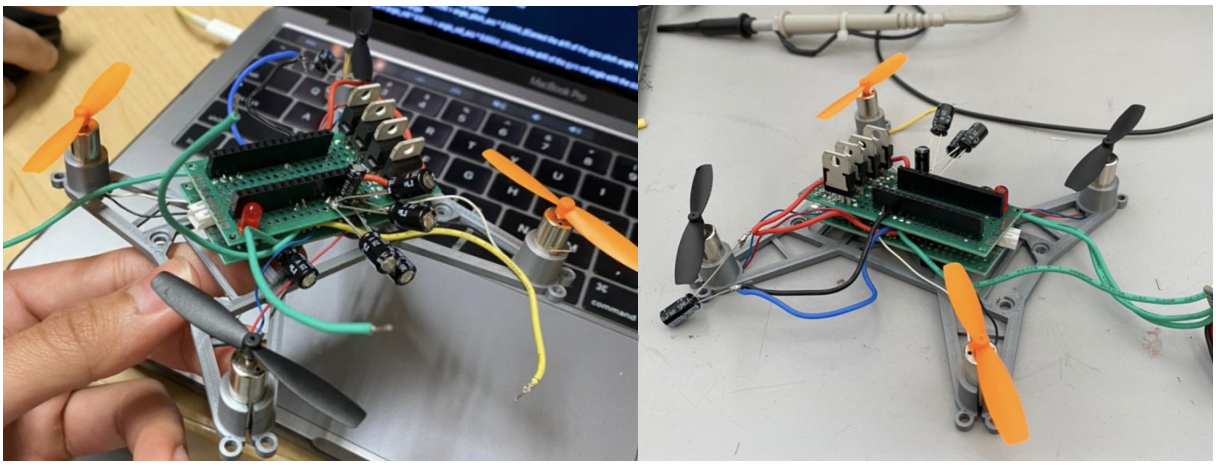


Figure 7: Built drone

Conclusion

The Nano drone wasn't perfectly completed by the deadline of the project but all the functionalities were presented and were successfully presented. The gyroscope and PID control was replicated on the breadboard while the application control over BLE was successfully tested using the Arduino Nano 33 BLE. The drone also worked when the battery was connected. This project had a strong relation to proof of concept as more functions can be implemented and integrated within the drone project.